RAPPORT DE PROJET

Facilo

Projet de développement web

Journal numérique de petites annonces

PLANIFICATION ET CONCEPTION D'UN PROJET WEB

Module : **Gestion d'un projet web**

Filière : Développement digital - ISTA NTIC, Tanger

Équipe

Mohamed Tarchouli	Product Owner
Ilias Yakhlef	Scrum Master
Abdellah Zemran	Développeur

Octobre 2026

Sommaire

1. Présentation du projet
2. Décomposition des tâches
3. Diagramme de Gantt
4. Diagramme de PERT et chemin critique
5. Organisation en modèle en V
6. Organisation Scrum avec Jira
7. Maquettes Figma
8. Développement du front-end
9. Back-end et base de données

1. Présentation du projet

Facilo est une application web de petites annonces. Une société spécialisée dans l'édition et la diffusion d'annonces (immobilier, emploi, formation, véhicules, multimédia...) distribue aujourd'hui ses journaux gratuitement en format papier. Pour diversifier son offre, elle souhaite créer un **journal numérique de petites annonces**.

Fonctionnalités attendues

- **Annonceurs** (particuliers ou professionnels) : inscription avec nom d'utilisateur, mot de passe et informations générales.
- **Annonces** libres d'accès au public, avec type, année, description, prix et photo.
- **Durée de vie** limitée à 1 mois, prolongeable à la demande de l'annonceur.
- **Catégories et sous-catégories** (immobilier, véhicules, multimédia...).
- **Recherche multicritères** : catégories, sous-catégories, dates, villes.
- Une application **ergonomique** qui donne un accès rapide et sélectif à l'information.

Objectif de ce rapport

Ce rapport présente la démarche suivie par l'équipe : décomposition du projet en tâches, planification (Gantt et PERT), organisation selon le modèle en V, suivi Scrum dans Jira, conception des maquettes sur Figma, puis réalisation du front-end.

2. Décomposition des tâches

Le projet est découpé en **34 tâches** sur un seul niveau, regroupées en 8 phases. Chaque tâche a une durée en jours ouvrés et des prédécesseurs (tâches à terminer avant de commencer). Le projet démarre le **07/09/2026** et les week-ends ne sont pas travaillés.

Les phases du projet

Phase	Tâches	Début	Fin	Jours (calendrier)	dont jours ouvrés
Analyse	1	07/09	09/09	3	3
Identité	2	10/09	10/09	1	1
Merise / BD	3 à 6	10/09	16/09	7	5
UML	7 à 11	10/09	16/09	7	5
UI / UX	12 à 14	10/09	15/09	6	4
Front-end	15 à 18	16/09	24/09	9	7
Back-end	19 à 28	18/09	28/09	11	7
Tests / Déploiement	29 à 32	29/09	02/10	4	4
Documentation	33 et 34	05/10	06/10	2	2

Les phases se chevauchent : la durée totale n'est donc pas la somme des phases. Le projet dure 30 jours calendaires, dont 22 jours ouvrés.

Liste complète des tâches

N°	Phase	Tâche	Durée	Début	Fin	Préd.	Marge
1	Analyse	Analyse du projet	3 j	07/09/2026	09/09/2026	-	0 j
2	Identité	Logo et slogan	1 j	10/09/2026	10/09/2026	1	8 j
3	Merise / BD	Dictionnaire de données	1 j	10/09/2026	10/09/2026	1	1 j
4	Merise / BD	MCD	2 j	11/09/2026	14/09/2026	3	1 j
5	Merise / BD	MLD	1 j	15/09/2026	15/09/2026	4	1 j
6	Merise / BD	Création de la base de données	1 j	16/09/2026	16/09/2026	5	1 j
7	UML	Diagramme de classes	1 j	10/09/2026	10/09/2026	1	1 j
8	UML	Diagramme de cas d'utilisation	1 j	11/09/2026	11/09/2026	7	1 j
9	UML	Diagramme de séquence	1 j	14/09/2026	14/09/2026	8	1 j
10	UML	Diagramme d'activité	1 j	15/09/2026	15/09/2026	9	1 j
11	UML	Diagramme de transition	1 j	16/09/2026	16/09/2026	10	1 j
12	UI / UX	Architecture des pages (Figma)	1 j	10/09/2026	10/09/2026	1	0 j
13	UI / UX	UI	2 j	11/09/2026	14/09/2026	12	0 j
14	UI / UX	UX	1 j	15/09/2026	15/09/2026	13	0 j
15	Front-end	Structure HTML	2 j	16/09/2026	17/09/2026	14	0 j
16	Front-end	Design avec CSS	2 j	18/09/2026	21/09/2026	2, 15	3 j
17	Front-end	Responsive design	1 j	22/09/2026	22/09/2026	16	3 j
18	Front-end	JavaScript / interactions	2 j	23/09/2026	24/09/2026	17	3 j
19	Back-end	PHP + connexion MySQL	1 j	18/09/2026	18/09/2026	6, 11, 15	0 j
20	Back-end	Inscription des annonceurs	1 j	21/09/2026	21/09/2026	19	0 j
21	Back-end	Authentification / déconnexion	1 j	22/09/2026	22/09/2026	20	0 j
22	Back-end	Catégories et sous-catégories	1 j	22/09/2026	22/09/2026	20	10 j
23	Back-end	Dépôt d'une annonce	1 j	23/09/2026	23/09/2026	21	0 j
24	Back-end	Upload des photos	1 j	24/09/2026	24/09/2026	23	0 j

N°	Phase	Tâche	Durée	Début	Fin	Préd.	Marge
25	Back-end	Consultation + détail des annonces	1 j	25/09/2026	25/09/2026	24	0 j
26	Back-end	Recherche multicritères	1 j	28/09/2026	28/09/2026	25	0 j
27	Back-end	Modification / suppression	1 j	25/09/2026	25/09/2026	24	1 j
28	Back-end	Durée de vie 1 mois + extension	1 j	28/09/2026	28/09/2026	25	0 j
29	Tests / Déploiement	Tests de la base de données	1 j	29/09/2026	29/09/2026	26, 27, 28	0 j
30	Tests / Déploiement	Tests fonctionnels et de recherche	1 j	30/09/2026	30/09/2026	18, 29	0 j
31	Tests / Déploiement	Corrections et validation finale	1 j	01/10/2026	01/10/2026	30	0 j
32	Tests / Déploiement	Hébergement / mise en ligne	1 j	02/10/2026	02/10/2026	31	0 j
33	Documentation	Documentation technique	1 j	05/10/2026	05/10/2026	32	0 j
34	Documentation	Préparation de la démonstration	1 j	06/10/2026	06/10/2026	33	0 j

Marge = retard possible sans retarder le projet. Une marge de 0 j signifie que la tâche est critique.

3. Diagramme de Gantt

Le diagramme de Gantt a été réalisé avec **GanttProject**. Chaque barre représente une tâche, positionnée selon sa date de début et sa durée. Les flèches sont les dépendances « fin - début » : une tâche ne commence qu'après la fin de son prédécesseur. Les colonnes grisées sont les week-ends.

Les couleurs distinguent les phases : **bleu** analyse, **rouge** identité visuelle, **vert** base de données, **violet** UML, **cyan** UI/UX, **jaune** front-end, **rose** back-end, **orange** tests et déploiement, **lilas** documentation.

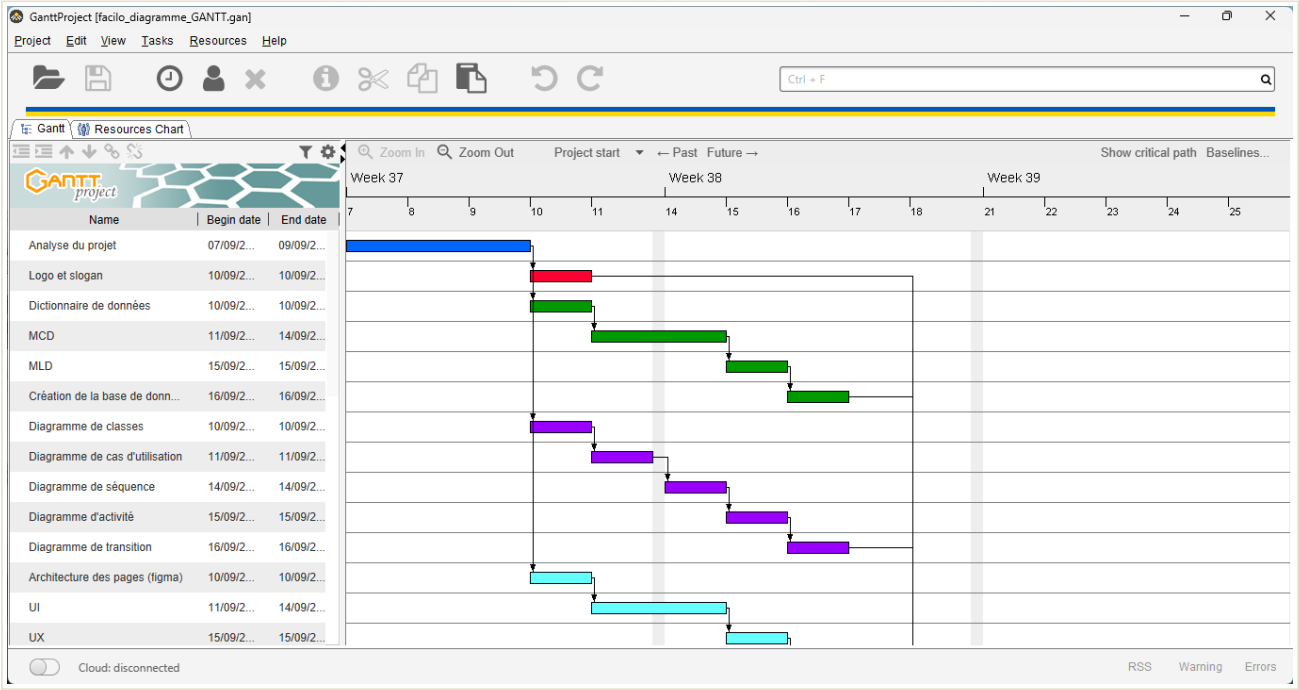


Figure 1 - Analyse, base de données (Merise), UML et début de l'UI/UX

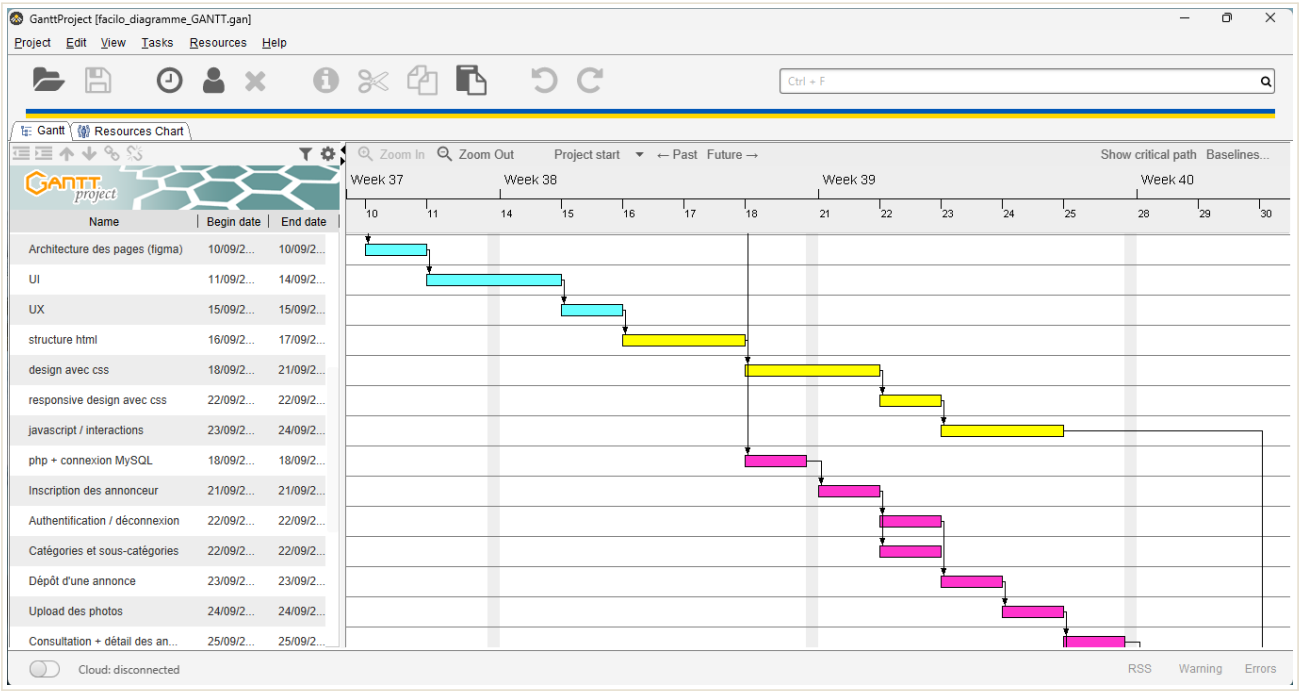


Figure 2 - UI/UX, développement front-end et début du back-end

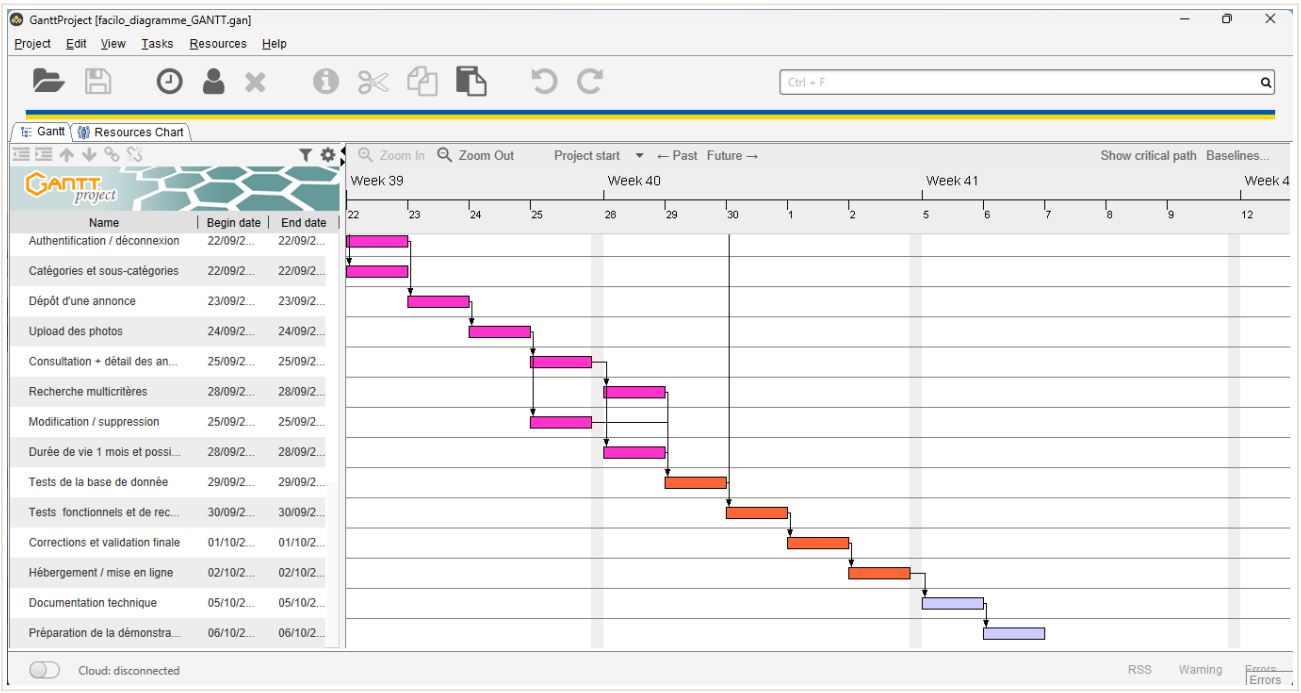


Figure 3 - Fin du back-end, tests, mise en ligne et documentation

4. Diagramme de PERT et chemin critique

Le diagramme de PERT montre l'enchaînement logique des tâches. Dans cette représentation, chaque **cercle** est une étape et chaque **flèche** est une tâche, avec son numéro et sa durée (par exemple *19 · 1 j*).

Comment lire un cercle

- En haut : le **numéro de l'étape**.
- En bas à gauche (bleu) : la **date au plus tôt**.
- En bas à droite (violet) : la **date au plus tard**.
- Une flèche **pointillée** est une tâche fictive de durée 0, qui sert à représenter une dépendance.
- Les flèches et cercles **rouges** forment le chemin critique : date au plus tôt = date au plus tard.

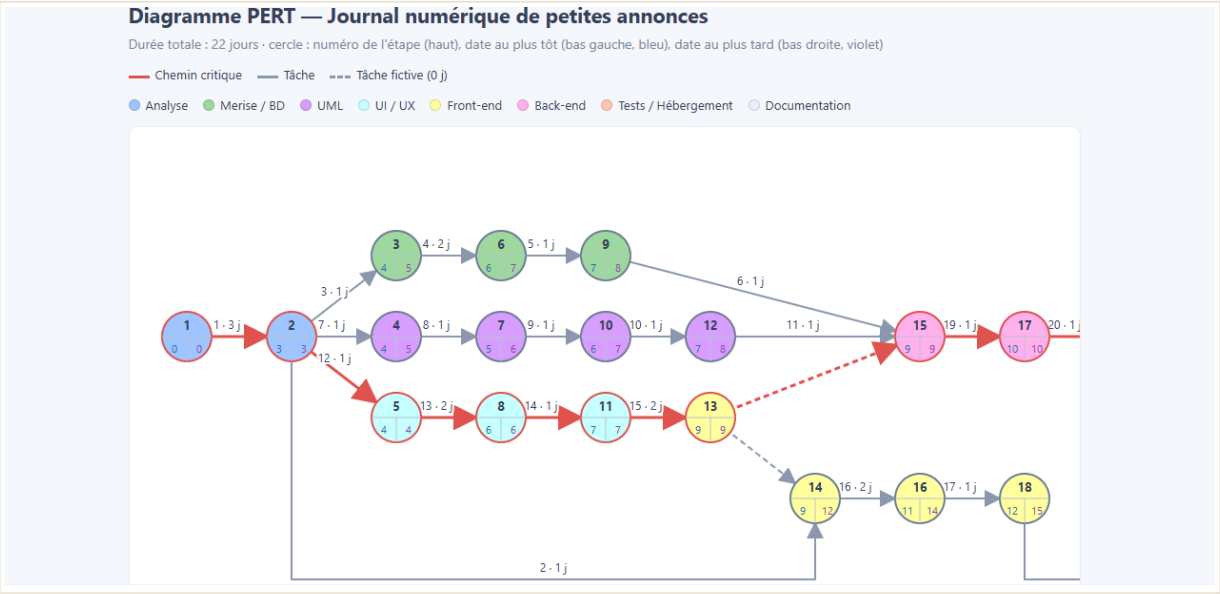


Figure 4 - PERT : de l'analyse au début du back-end (étapes 1 à 18)

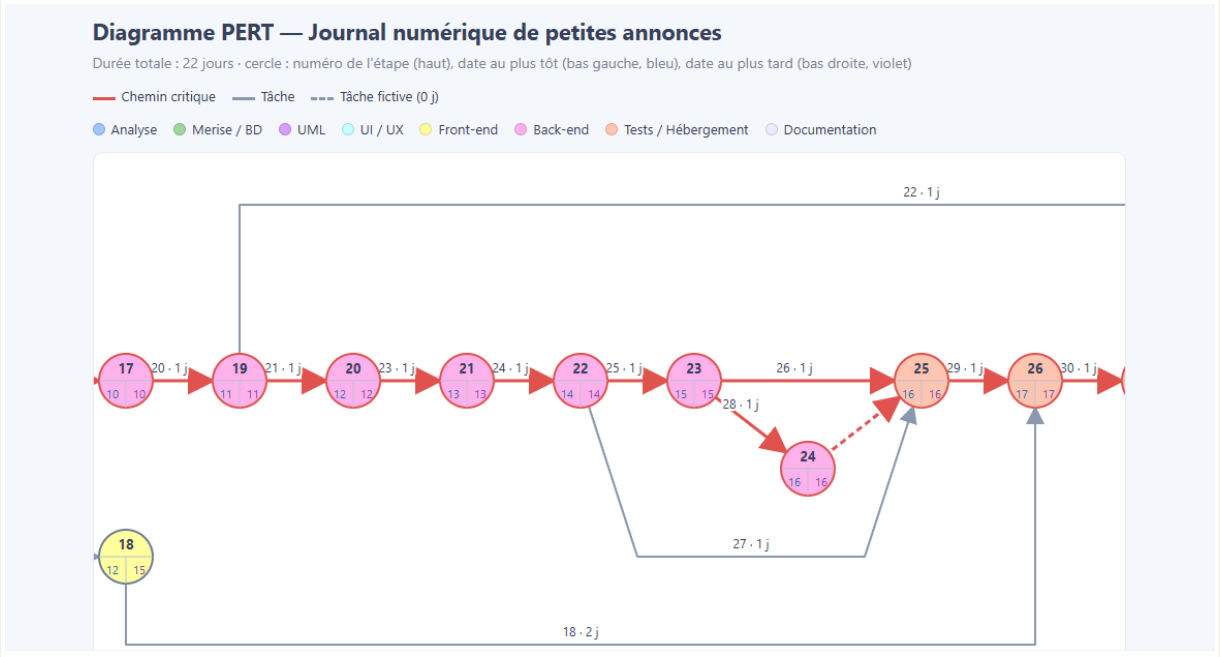


Figure 5 - PERT : cœur du back-end (étapes 17 à 26)

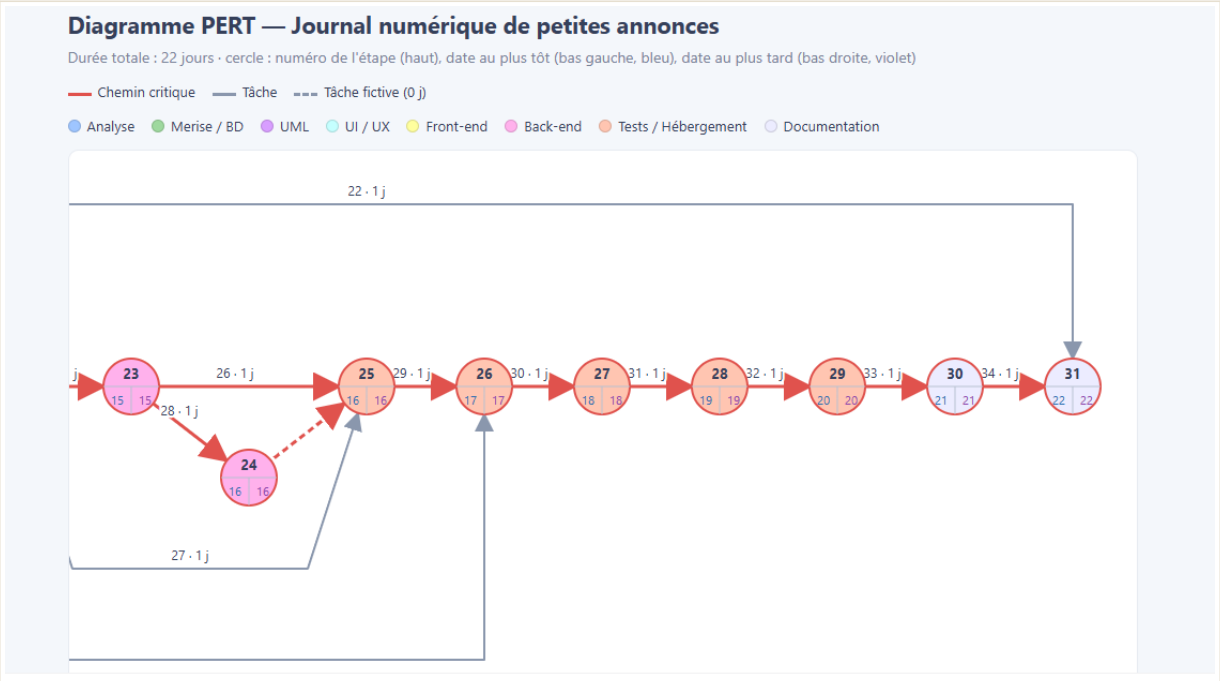


Figure 6 - PERT : fin du back-end, tests, mise en ligne et documentation (étapes 23 à 31)

Chemin critique

Le chemin critique est la suite de tâches sans marge : tout retard sur l'une d'elles retarde la fin du projet. La **durée minimale de réalisation est de 22 jours ouvrés**.

1 - 12 - 13 - 14 - 15 - 19 - 20 - 21 - 23 - 24 - 25 - (26 ou 28) - 29 - 30 - 31 - 32 - 33 - 34

Il y a **deux chemins critiques** qui ne diffèrent que par une tâche : « Recherche multicritères » (26) ou « Durée de vie » (28), de 1 jour chacune, toutes deux entre la consultation (25) et les tests de la base de données (29). Au total, 19 tâches sont critiques.

Tâches non critiques

Tâches	Marge	Remarque
2 - Logo et slogan	8 j	Nécessaire seulement avant le design CSS
22 - Catégories et sous-catégories	10 j	Peut être réalisée en parallèle du reste du back-end
16, 17, 18 - CSS, responsive, JavaScript	3 j	Le front-end a de la marge avant les tests fonctionnels
3 à 11 - Merise et UML	1 j	Presque critiques : 1 jour de marge seulement
27 - Modification / suppression	1 j	Parallèle à la consultation et à la recherche

5. Organisation en modèle en V

Le **modèle en V** relie chaque phase de conception (branche descendante) au test qui la vérifie (branche montante). Il convient bien à ce projet : les besoins sont clairs dès le départ et les phases se suivent dans un ordre logique. Le bas du V est la réalisation du code.

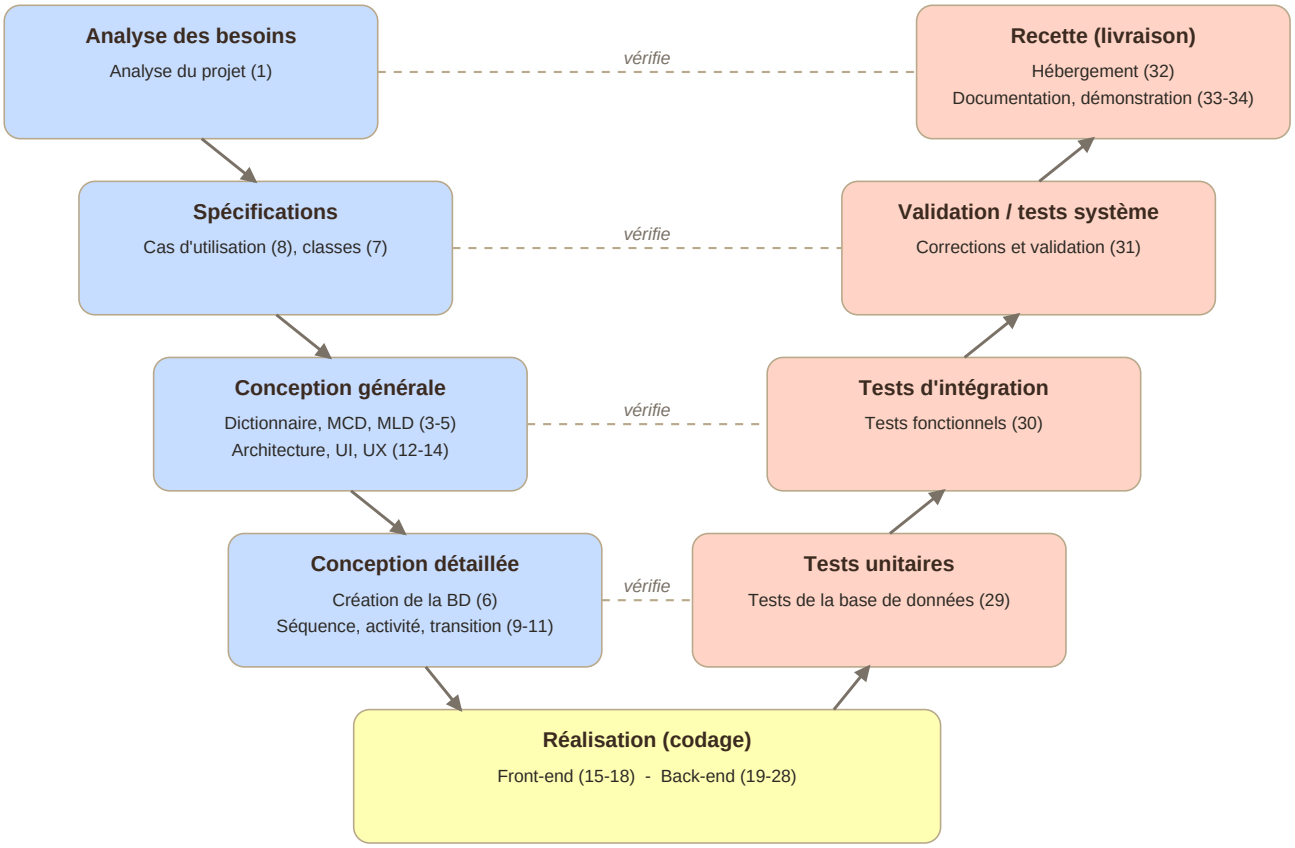


Figure 7 - Les tâches du projet placées dans le modèle en V (numéros de tâches entre parenthèses)

Chaque niveau de gauche est contrôlé par le niveau de droite : les spécifications sont validées par les tests système, la conception générale par les tests d'intégration et la conception détaillée par les tests unitaires. Il n'y a pas de tâche dédiée aux tests unitaires de chaque module : ils sont faits pendant le développement.

6. Organisation Scrum avec Jira

L'équipe suit la méthode **Scrum**, avec un espace de travail **Jira** nommé « facilo project ». Les rôles sont répartis ainsi :

- **Mohamed Tarchouli** - Product Owner : définit les besoins et les priorités.
- **Ilias Yakhlef** - Scrum Master : veille au bon déroulement de la méthode.
- **Abdellah Zemran** - Développeur.

Backlog et user stories

Les tâches de la planification ont été transformées en **epics** (grandes parties) et en **user stories** (fonctionnalités). Chaque story porte un nombre de **story points** (3 ou 5) qui estime son effort, une priorité et un statut. Le sprint 4, visible ci-dessous, regroupe les stories du back-end et de la base de données : inscription et authentification des annonceurs, dépôt d'annonce, upload de photos, consultation, recherche multicritères, gestion des catégories, modification / suppression et durée de vie des annonces.

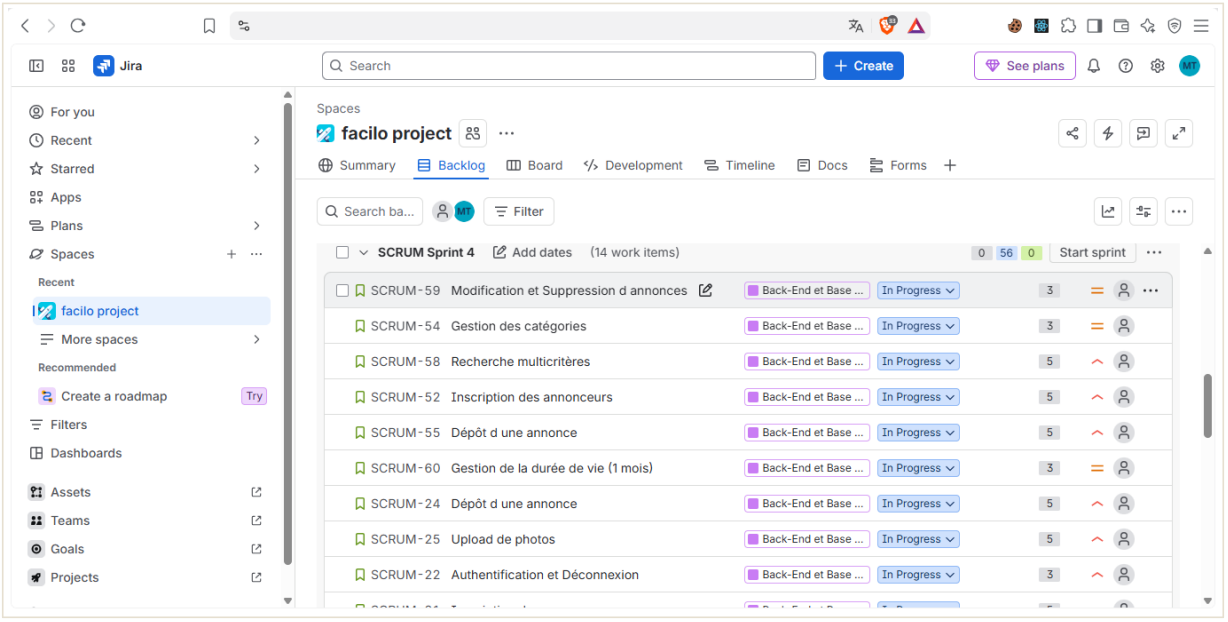


Figure 8 - Backlog Jira : sprint 4 (14 éléments), epic « Back-End et Base de données »

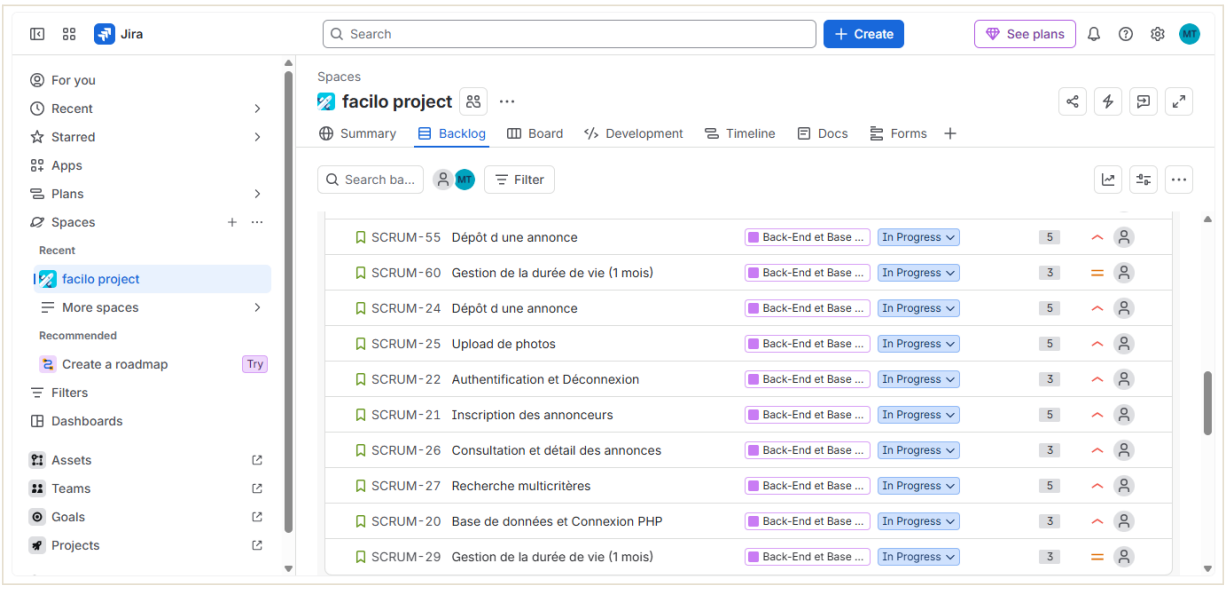


Figure 9 - Backlog Jira : stories du back-end avec points et priorités

Timeline des epics

La vue Timeline suit l'avancement par epic : *Analyse et Conception*, *Front-End et Design*, *Back-End et Base de données*, *Tests et Déploiement* et *Documentation*. Les barres vertes correspondent aux epics terminés, la barre bleue à l'epic en cours (back-end) et les barres grises aux epics à venir. Ces epics reprennent les phases du Gantt.

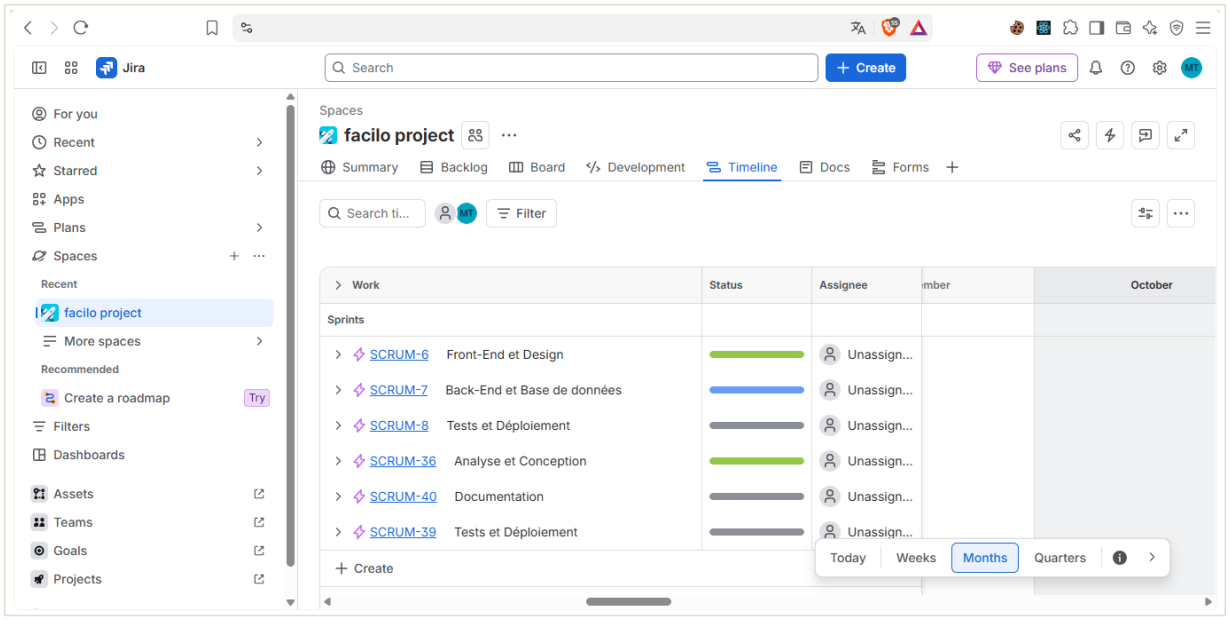


Figure 10 - Timeline Jira des epics du projet

7. Maquettes Figma

Avant de coder, les écrans ont été dessinés sur **Figma** (tâches 12 à 14 du Gantt : architecture des pages, UI et UX). La maquette est conçue pour mobile et reprend le logo **Facilo** et le slogan « cherche-la, trouve-la et prends-le » dans une palette de beige et de brun.



Figure 11 - Écrans mobiles du prototype Figma

Les écrans

- **Écran d'accueil** avec le logo et le slogan.
- **Bienvenue** : trois accès - continuer comme utilisateur, continuer comme visiteur, ou accès administrateur.
- **Choix inscription / connexion** pour l'utilisateur.
- **Inscription** : nom, prénom, e-mail et numéro avec envoi d'un code de vérification.
- **Connexion** : e-mail et mot de passe.
- **Accès administrateur** protégé par un code.
- **Page d'accueil** : barre de recherche, catégories et section « Spotlights ».

8. Développement du front-end

Le front-end suit le design validé : même nom, même identité visuelle (marron et or) et une interface sombre avec un bouton pour changer de thème. Il correspond aux tâches 15 à 18 du Gantt (structure HTML, CSS, responsive, JavaScript).

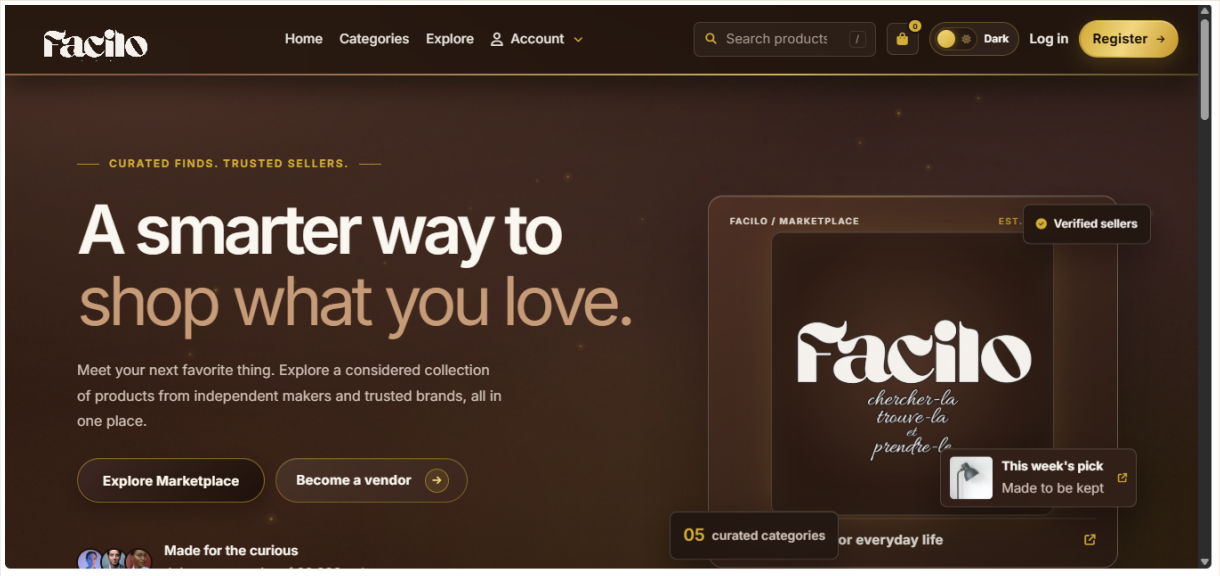


Figure 12 - Page d'accueil : navigation, recherche, panier, thème clair / sombre et boutons d'action

- **Barre de navigation** : Home, Categories, Explore, menu Account, recherche (raccourci « / »), panier avec compteur, bascule Dark / Light, Log in et Register.
- **Section d'accueil** : titre, texte de présentation et deux boutons, « Explore Marketplace » et « Become a vendor ».
- **Éléments de confiance** : badge « Verified sellers » et sélection de la semaine.

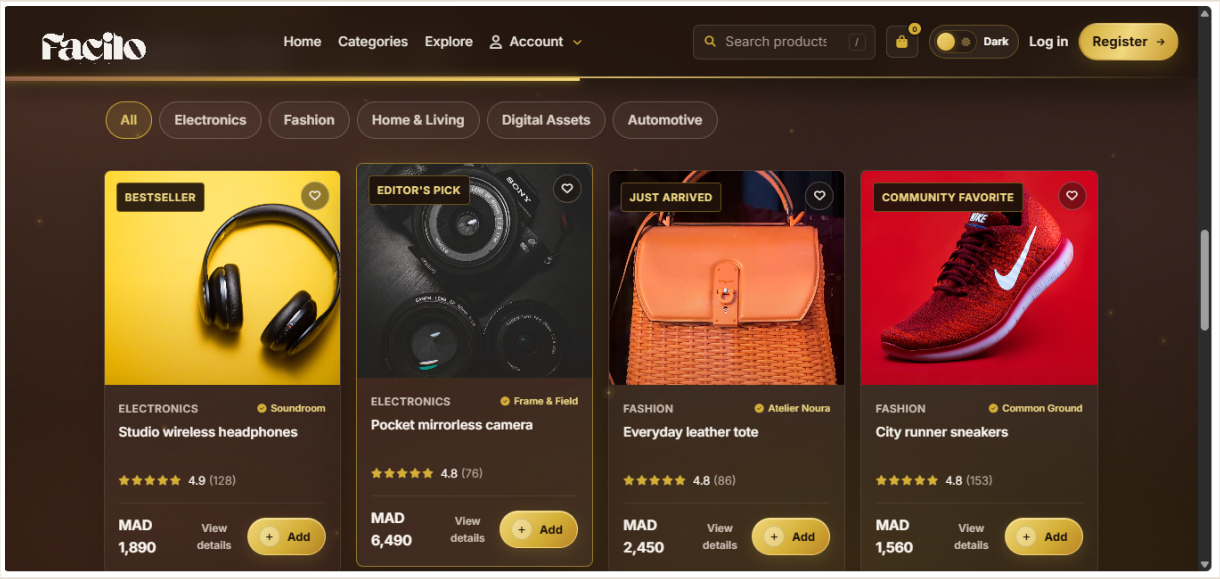


Figure 13 - Catalogue : filtres par catégorie et cartes de produits

- **Filtres** par catégorie : All, Electronics, Fashion, Home & Living, Digital Assets, Automotive.
- **Cartes de produit** : badge, vendeur vérifié, nom, note, prix en MAD, boutons « View details » et « Add ».

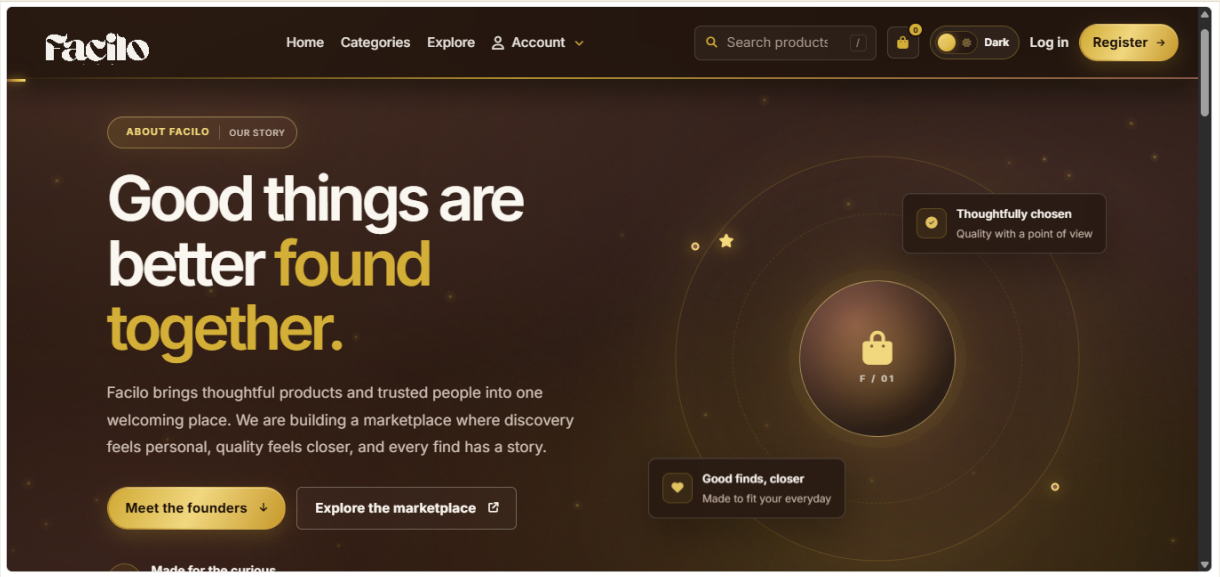


Figure 14 - Page « About Facilo » : présentation du projet et de l'équipe

La page de présentation raconte l'histoire de Facilo et propose deux actions : « Meet the founders » et « Explore the marketplace ».

9. Back-end et base de données

Après le front-end, l'équipe a ajouté le **back-end** du projet, puis a relié la base de données aux services cloud **Aiven** (base MySQL) et **Cloudinary** (images). Cette partie correspond à la phase back-end du Gantt (tâches 19 à 28) : connexion MySQL, inscription, authentification, dépôt d'annonces, upload des photos, consultation, recherche, modification et durée de vie.

Étape 1 - Ajout du back-end au projet Facilo

Le back-end est écrit en **PHP** et placé dans le même dépôt que le front-end (HTML, CSS, JavaScript). Il est organisé en dossiers, visibles dans la Figure 15 :

- **api/** : les points d'entrée appelés par le front-end - ads.php (annonces), auth.php (authentification), contact.php (contact), favorites.php (favoris), profile.php (profil) et sellers.php (vendeurs).
- **includes/** : le code partagé par ces points d'entrée (authentification, initialisation, Cloudinary, réponses de l'API, validation des données).
- **config/db.php** : la configuration de la connexion à la base de données.
- **certs/cert.pem** : le certificat utilisé pour la connexion sécurisée (SSL) à la base.
- **composer.json** : déclare deux bibliothèques PHP, **cloudinary/cloudinary_php** (^3.0) pour envoyer les images sur Cloudinary et **firebase/php-jwt** (^6.10) pour gérer les jetons JWT d'authentification.

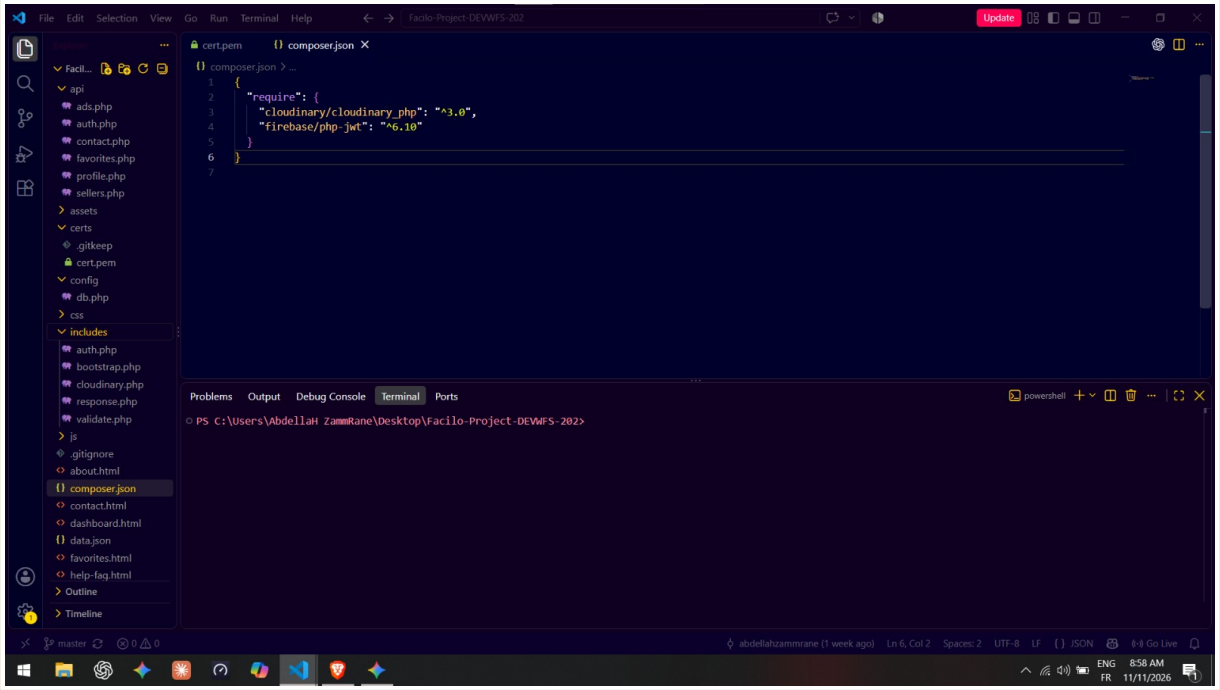


Figure 15 - Structure du projet dans Visual Studio Code : dossiers api, config, certs et includes, fichier composer.json

Le projet est versionné sur **GitHub** (dépôt Facilo-Project-DEVWFS-202) et partagé entre 4 contributeurs. La Figure 16 montre les commits liés au back-end : « addition the back-end » pour api et includes, « Adding my backend » pour config, « Adding the cert.pem » pour certs et « test deploy » pour composer.json et composer.lock. PHP apparaît maintenant dans les langages du dépôt (3,5 %).

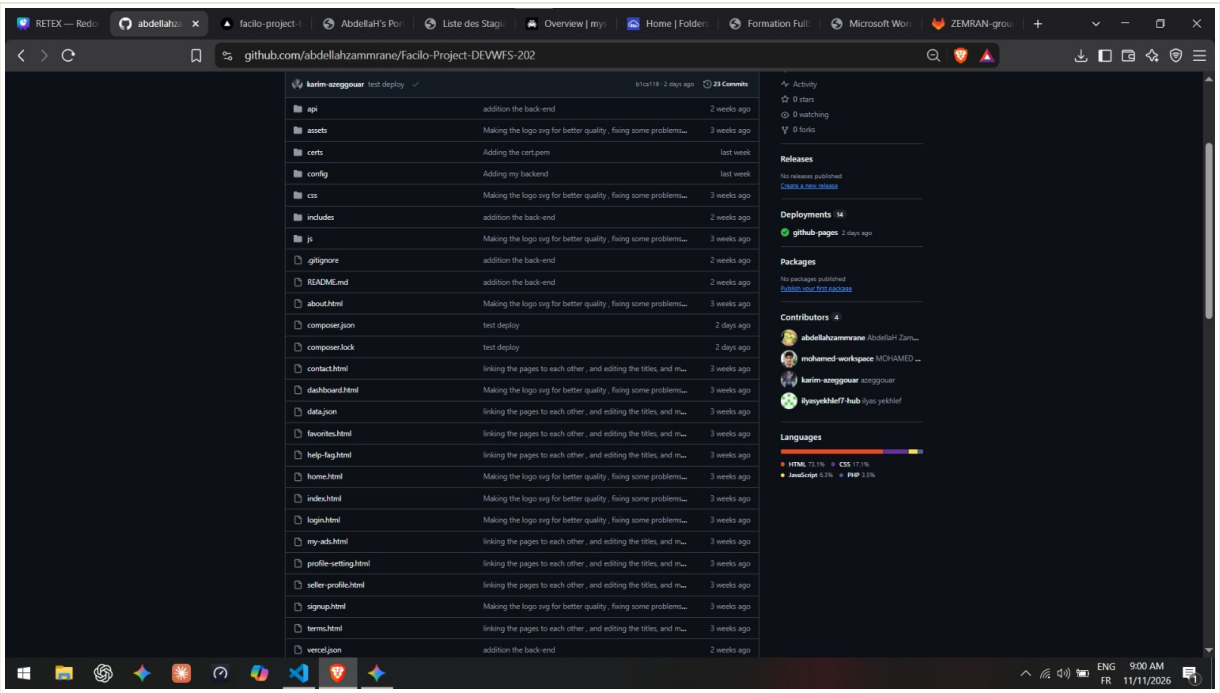


Figure 16 - Dépôt GitHub du projet après l'ajout du back-end

Étape 2 - Connexion de la base de données à Aiven et Cloudinary

Base MySQL sur Aiven. La base de données est une base **MySQL 8.4** hébergée dans le cloud chez Aiven. La connexion se fait depuis le terminal avec le client mysql : on indique l'hôte fourni par Aiven, le port (17036), l'utilisateur (avnadmin), la base (defaultdb) et l'option **--ssl-mode=REQUIRED**, qui impose une connexion chiffrée. La commande **SHOW DATABASES** liste ensuite les bases disponibles et **USE defaultdb** sélectionne celle du projet.

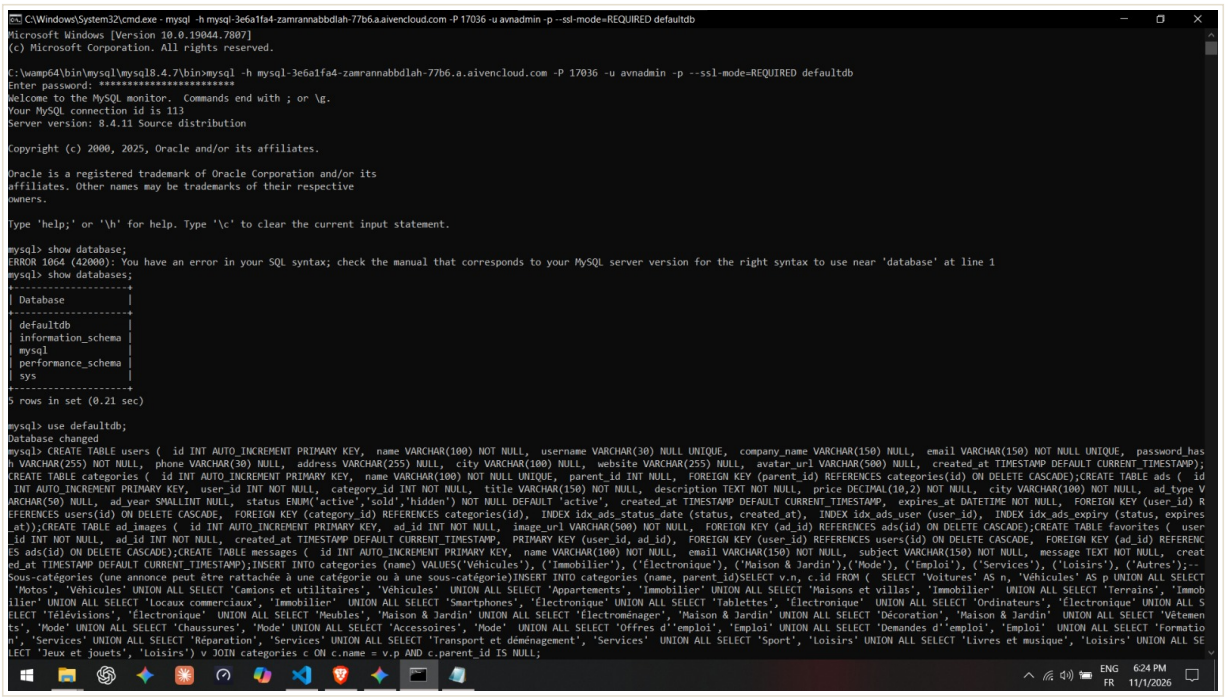


Figure 17 - Connexion à la base MySQL d'Aiven en SSL, puis création des tables avec le script SQL

Le script SQL est ensuite exécuté : il crée six tables et insère les catégories (Véhicules, Immobilier, Électronique, Maison & Jardin, Mode, Emploi, Services, Loisirs, Autres) avec leurs sous-catégories.

Table	Rôle	Colonnes principales
users	Annonceurs	name, username, company_name, email, password_hash, phone, address, city, website, avatar_url, created_at
categories	Catégories et sous-catégories	name, parent_id (lien vers la catégorie parente)
ads	Annonces	user_id, category_id, title, description, price, city, ad_type, ad_year, status, created_at, expires_at
ad_images	Photos des annonces	ad_id, image_url
favorites	Favoris des utilisateurs	user_id, ad_id, created_at (clé primaire composée)
messages	Messages du formulaire de contact	name, email, subject, message, created_at

La durée de vie d'un mois demandée dans le cahier des charges est gérée par la colonne **expires_at** de la table ads, indexée avec le statut de l'annonce (idx_ads_expiry). Les annonces existantes reçoivent 30 jours à partir de l'exécution du script.

Pour vérifier le résultat, les commandes **DESCRIBE users** et **DESCRIBE categories** affichent la structure des tables : 12 champs pour users (clé primaire id, username et email uniques) et 3 champs pour categories (id, name, parent_id). La commande ALTER TABLE relancée ensuite renvoie l'erreur « Duplicate column name » pour username, parent_id et ad_type : ces colonnes existent déjà, la structure est donc en place (Figure 18).

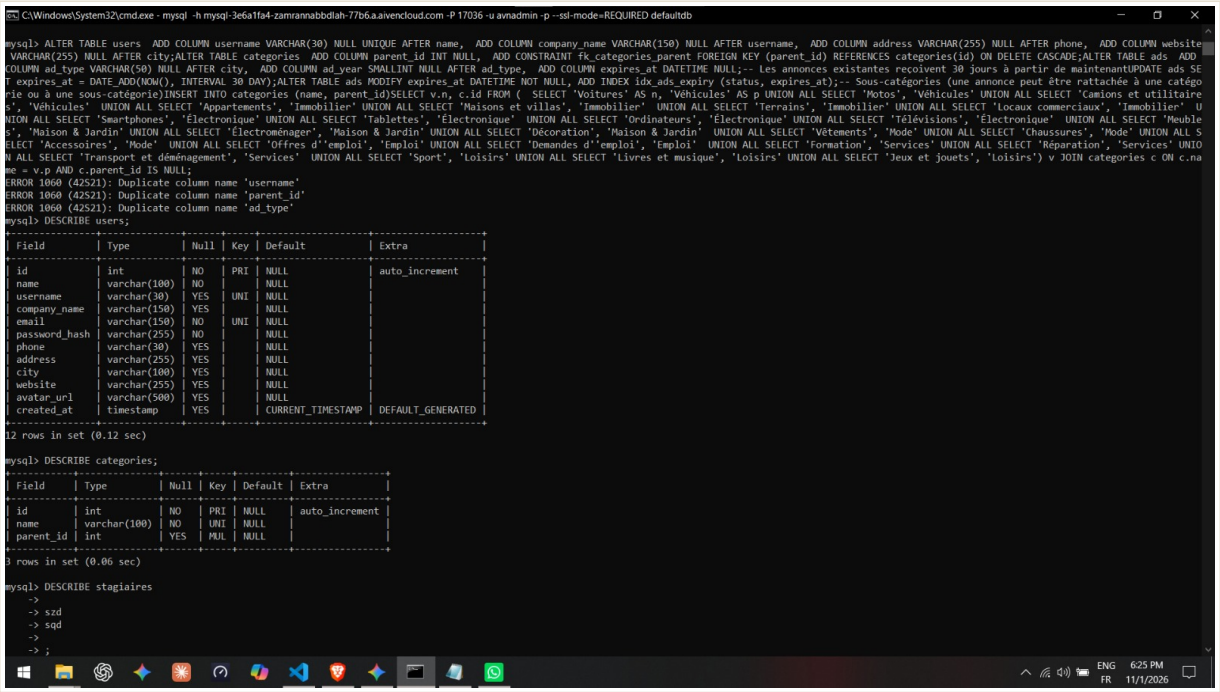


Figure 18 - Vérification de la structure des tables avec DESCRIBE

Images sur Cloudinary. Les photos des annonces et les avatars ne sont pas enregistrés dans MySQL : le fichier est envoyé sur Cloudinary par la bibliothèque PHP (includes/cloudinary.php) et la base garde seulement son adresse (URL), dans les colonnes **ad_images.image_url** et **users.avatar_url**. Cela correspond à la tâche 24 du Gantt (upload des photos).

Résultat. Le back-end PHP est relié à une base MySQL hébergée sur Aiven et à Cloudinary pour les images. L'étape suivante est de relier le front-end à ce back-end.